

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Информационные технологии»
Тема: «Алгоритмы и структуры данных в Python.»

Студент гр. 5387

Степанов Леонид
Артемович

Преподаватель

Клюканов Виталий
Константинович

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Цель работы	3
Задание	4
Основные теоретические положения	5
Выполнение работы	7
Тестирование	9
Выводы	10
ПРИЛОЖЕНИЕ А	11

Цель работы

Изучение базовых структур данных и алгоритмов на языке Python.
Приобретение практических навыков реализации связанного однонаправленного списка, управления памятью (ссылками) и анализа вычислительной сложности алгоритмов.

Задание

Реализовать связный однонаправленный список, состоящий из двух зависимых классов: Node (элемент списка) и LinkedList (сам список). Обеспечить методы для добавления, удаления, изменения элементов, а также строкового представления списка. В отчете дать теоретическую сводку об отличиях связного списка от массива, указать сложность каждого реализованного метода и описать специфику бинарного поиска в связном списке.

Основные теоретические положения

1. Понятие связного списка и отличия от массива

Связный список — это базовая динамическая структура данных, состоящая из узлов, каждый из которых содержит сами данные и ссылку (указатель) на следующий узел в последовательности.

Основные отличия связного списка от массива (классического списка Python, реализованного как динамический массив):

- **Выделение памяти:** Элементы массива хранятся в памяти непрерывным блоком. Узлы связного списка разбросаны в памяти произвольно, связь поддерживается только за счет указателей.
- **Доступ к элементам:** В массиве возможен произвольный доступ к любому элементу по индексу за $O(1)$. В связном списке доступ строго последовательный — чтобы добраться до n -го элемента, нужно пройти от начала списка шагов n , что занимает времени $O(n)$.
- **Вставка и удаление:** В массиве вставка или удаление в начале или середине требует сдвига всех последующих элементов, что занимает времени $O(n)$. В связном списке, если указатель уже находится на нужном месте, вставка и удаление происходят за путем переопределения ссылок.

2. Вычислительная сложность реализованных методов

- **Класс Node:**
 - `__init__` —
 - `get_data` —
 - `change_data` —
 - `__str__` —
- **Класс LinkedList:**
 - `__init__` —
 - `__len__` —

- `append` — . В текущей реализации алгоритм проходит весь список от головы (`head`) до конца (пока `curr.next` не станет `None`), чтобы добавить новый элемент.
- `__str__` — , так как требуется обойти весь список для формирования строки.
- `pop` — . Для удаления последнего элемента необходимо найти предпоследний узел, что требует обхода всего списка.
- `change_on_start` — . Алгоритм проходит шагов от начала списка для поиска нужного элемента.
- `clear` — , так как достаточно обнулить ссылку на `head` и длину списка (сборщик мусора Python сам удалит недоступные узлы).

3. Бинарный поиск в связном списке

Классический алгоритм бинарного поиска требует произвольного доступа к элементам коллекции для вычисления среднего элемента на каждом шаге (в массиве это делается за $O(1)$). В однонаправленном связном списке найти элемент посередине можно только последовательным перебором, что занимает времени на каждой итерации. Из-за этого общая временная сложность бинарного поиска в связном списке вырождается и становится хуже линейного поиска — $O(n)$. По этой причине бинарный поиск для связного списка на практике не применяется; вместо него используют обычный линейный поиск за $O(n)$ или другие структуры данных (например, бинарные деревья поиска). В стандартном списке Python (массиве) бинарный поиск работает эффективно за $O(\log n)$.

Выполнение работы

Класс Node Описывает узел связного списка.

- **Поля:** `data` (хранимое значение) и `next` (ссылка на следующий узел).
- **Методы:** *
 - **`__init__(data, next=None)`:** Инициализирует узел. Аргументы: `data` — хранимое значение узла; `next` — ссылка на следующий узел (по умолчанию `None`).
 - **`get_data()`:** Возвращает значение поля `data`. Аргументов не принимает. **`change_data(new_data)`:** Устанавливает новое значение поля `data`. Аргументы: `new_data` — новое значение для замены текущего.
 - **`__str__()`:** Возвращает отформатированную строку с информацией о текущих данных и данных следующего узла. Аргументов не принимает.

Класс LinkedList Описывает структуру данных «Связный список».

- **Поля:** `head` (ссылка на первый узел) и `length` (текущее количество узлов).
- **Методы:**
 - **`__init__(head=None)`:** Конструктор. Создает пустой список или список из одного элемента. Аргументы: `head` — начальный узел типа `Node` (по умолчанию `None`; если передан, `length` устанавливается в 1).
 - **`__len__()`:** Возвращает актуальную длину списка, хранящуюся в поле `length`. Аргументов не принимает.
 - **`append(element)`:** Добавляет новый элемент в конец списка. Аргументы: `element` — добавляемое значение. Если список пуст, созданный узел становится головой; иначе выполняется итерация до последнего узла.
 - **`__str__()`:** Проходит по всем узлам и формирует итоговую строку согласно заданному формату. Аргументов не принимает.

- **pop():** Удаляет последний элемент списка. Аргументов не принимает. При попытке удаления из пустого списка генерирует исключение `IndexError`.
- **change_on_start(n, new_data):** Изменяет данные n-го элемента с начала списка (нумерация с 1). Аргументы: `n` — порядковый номер узла; `new_data` — новое значение. Если передан некорректный индекс ($n \leq 0$ или $n > \text{length}$), генерирует `KeyError`.
- **clear():** Очищает список, сбрасывая `head` в `None` и `length` в `0`. Аргументов не принимает.

Тестирование

Тестирование проводилось вручную путём последовательного вызова методов классов `Node` и `LinkedList` и проверки результатов на соответствие ожидаемым значениям.

Тест 1. Создание пустого списка. Создан объект `LinkedList()`. Проверено, что `len(lst) == 0` и `str(lst) == "LinkedList[]"`. Результат: тест пройден.

Тест 2. Добавление элементов. После вызовов `lst.append(1)`, `lst.append(2)`, `lst.append(3)` проверено, что `len(lst) == 3` и вывод содержит все три значения в правильном порядке. Результат: тест пройден.

Тест 3. Удаление последнего элемента (`pop`). После `lst.pop()` проверено, что `len(lst) == 2` и третий элемент отсутствует. При вызове `pop()` на пустом списке выброшено исключение `IndexError`. Результат: тест пройден.

Тест 4. Изменение элемента (`change_on_start`). После `lst.change_on_start(1, 99)` значение первого узла изменено на 99. При передаче некорректного индекса (0 или $> \text{length}$) выброшен `KeyError`. Результат: тест пройден.

Тест 5. Очистка списка (`clear`). После `lst.clear()` проверено, что `len(lst) == 0`. Результат: тест пройден.

Все тесты успешно пройдены; реализация методов соответствует заданию.

Выводы

Цель лабораторной работы достигнута: базовые структуры данных и алгоритмы изучены, связный однонаправленный список реализован на языке Python. В результате выполнения лабораторной работы были закреплены знания о динамических структурах данных и освоены навыки реализации однонаправленного связного списка на языке Python. Были реализованы классы Node и LinkedList с основными методами: добавлением элемента (append), извлечением (pop), очисткой (clear) и изменением элемента (change_on_start). В ходе работы был проведен анализ асимптотической сложности реализованных методов. Также была изучена специфика работы алгоритма бинарного поиска: было установлено, что из-за отсутствия произвольного доступа к памяти за $O(n)$, бинарный поиск неэффективен для связных списков в отличие от массивов (классических списков Python).

ПРИЛОЖЕНИЕ А

Исходный код программы (Файл main.py)

```
class Node:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next

    def get_data(self):
        return self.data

    def change_data(self, new_data):
        self.data = new_data

    def __str__(self):
        next_data = self.next.data if self.next else None
        return f"data: {self.data}, next: {next_data}"

class LinkedList:
    def __init__(self, head=None):
        self.head = head
        self.length = 1 if head else 0

    def __len__(self):
        return self.length

    def append(self, element):
        new_node = Node(element)

        if self.head is None:
            self.head = new_node
        else:
            curr = self.head
            while curr.next:
                curr = curr.next
            curr.next = new_node

        self.length += 1
```

```

def __str__(self):
    if self.length == 0:
        return "LinkedList[]"

    result = []
    curr = self.head

    while curr:
        next_data = curr.next.data if curr.next else None
        result.append(f"data: {curr.data}, next: {next_data}")
        curr = curr.next

    return f"LinkedList[length = {self.length}, [{';'.join(result)}]]"

def pop(self):
    if self.length == 0:
        raise IndexError("LinkedList is empty!")

    if self.length == 1:
        self.head = None
    else:
        curr = self.head
        while curr.next.next:
            curr = curr.next
        curr.next = None

    self.length -= 1

def change_on_start(self, n, new_data):
    if n <= 0 or n > self.length:
        raise KeyError("Element doesn't exist!")

    curr = self.head
    for _ in range(n - 1):
        curr = curr.next

```

```
curr.data = new_data

def clear(self):
    self.head = None
    self.length = 0
```